

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**

Department of Computer Science and Engineering

**CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise**

|                      |                 |                      |                                      |
|----------------------|-----------------|----------------------|--------------------------------------|
| <b>Course Code:</b>  | CSA10 / CSA1063 | <b>Course Title:</b> | Software Engineering                 |
| <b>CO Assessed:</b>  | CO4             | <b>Assessment:</b>   | AT2 – CI/CD Pipeline Design Exercise |
| <b>Weightage:</b>    | 20%             | <b>Total Marks:</b>  | 25                                   |
| <b>Student Name:</b> | Y Praveen Raj   | <b>Reg. No.:</b>     | 192372306                            |
| <b>Date:</b>         | 21-08-2026      | <b>Duration:</b>     | 60 minutes                           |

**CO4 Statement:** Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics.

**Code of Conduct**

I Y Praveen Raj (192372306) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sunny (Praveen)

**Assigned Problem Statement**

**Formal Verification Framework for E-Transaction Protocols Using Product Automata and Reachability Analysis for Fraud Detection**

**1. Project Requirements and CI/CD Needs Analysis**

The system under consideration has two deployable components: (a) a Python-based verification engine that constructs per-protocol automata, composes them into a product automaton, and runs CTL/reachability checks for replay attacks, double-spending, identity forgery, and race conditions; and (b) a web dashboard that visualises automata diagrams, verification results, and fraud-coverage reports. Because the engine's correctness is safety-critical — an undetected regression could mean a real fraud pattern is silently missed — the CI/CD needs go beyond typical web-app delivery:

- Every change to the automaton-construction or reachability-analysis logic must be automatically tested against a fixed regression suite of known attack scenarios before it can reach production.
- Builds must be fully reproducible (pinned dependency versions, containerised) since floating-point/graph-library version drift can silently change reachability results.
- Security scanning is mandatory given the system processes representations of financial transaction protocols.
- A human approval gate is required before production release, since this is a verification tool whose own correctness underpins fraud-detection guarantees.
- Fast feedback on pull requests (build + unit tests) is needed to keep the small student/team development loop productive, while heavier checks (E2E, load, security) run before merge to main.

**2. Pipeline Architecture and Workflow**

The pipeline follows the standard commit-to-deployment stages, adapted to the project's layered architecture (data layer → automaton engine → REST API → dashboard, as established in the project's CO2 architecture design). Every push to a feature

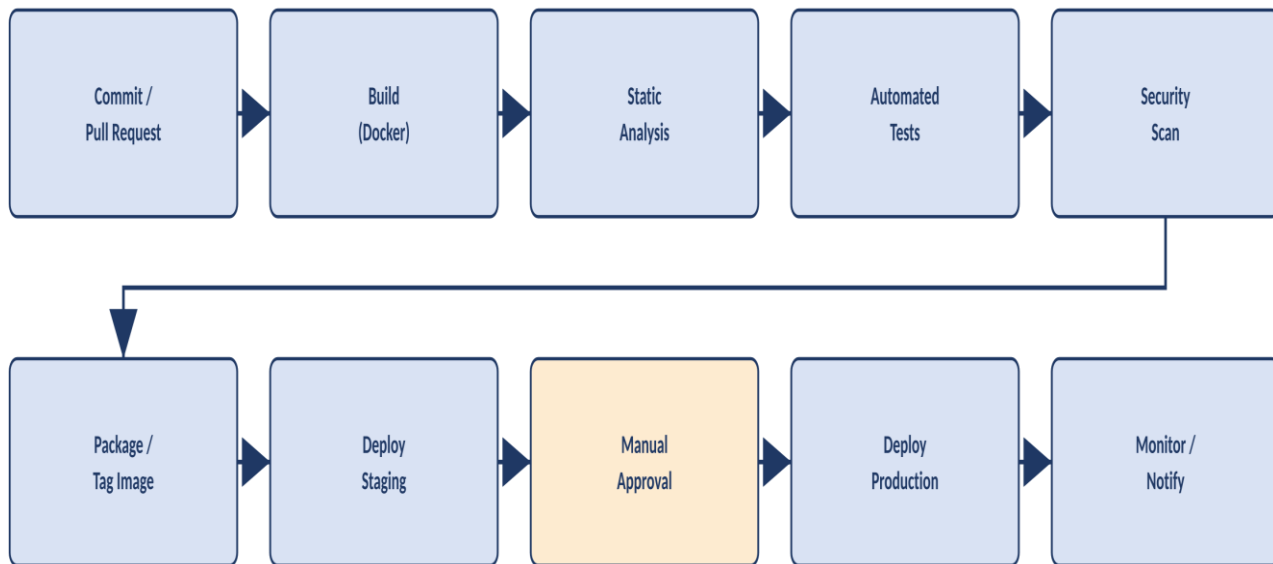
branch runs Stages 1–4 (fast feedback); merges to main additionally run Stages 5–10 (full release path with a manual gate before production).

| Stage                                     | Purpose                                                                                                    | Tools / Technology                                                                        | Justification                                                                                                                     |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1. Source Control                         | Trigger the pipeline on every commit/PR to the verification-engine and dashboard repositories              | Git, GitHub, GitHub Actions (webhook trigger)                                             | Branch protection on main; feature branches trigger CI, main-branch merges trigger CD                                             |
| 2. Build                                  | Install dependencies and compile/package the Python verification engine and the React/Node dashboard       | Python 3.11 + pip/venv, Node.js 20 + npm, Docker multi-stage build                        | Containerised build guarantees reproducibility of automata-construction dependencies (NetworkX/ Z3) across environments           |
| 3. Static Analysis & Code Quality         | Catch style, type, and security issues before tests run                                                    | flake8 / pylint & mypy (Python), ESLint (JS/TS), SonarCloud, Bandit, npm audit            | Automata and reachability logic is safety-critical; static typing and linting reduce logic errors early and cheaply               |
| 4. Automated Testing                      | Validate correctness of automaton construction, product-automaton composition, and reachability/CTL checks | PyTest with coverage.py (unit + integration), Selenium/Playwright (UI/E2E), locust (load) | Layered tests mirror the layered architecture (data → automaton engine → API → dashboard) established in the project's CO2 design |
| 5. Security Scanning                      | Detect vulnerable dependencies and container misconfigurations before deployment                           | Trivy (container scan), OWASP Dependency-Check, GitHub CodeQL                             | A fraud-detection framework must itself be free of exploitable vulnerabilities before being trusted with transaction data         |
| 6. Packaging                              | Produce a versioned, immutable deployment artefact                                                         | Docker image build & tag (semantic version + git SHA), push to GitHub Container Registry  | Immutable, tagged images allow exact rollback to any previously verified build                                                    |
| 7. Deploy – Staging                       | Deploy to an environment that mirrors production for smoke and regression testing                          | Docker Compose / Kubernetes (staging namespace), automated smoke tests                    | Confirms the automaton engine and dashboard interoperate correctly before real use                                                |
| 8. Manual Approval Gate                   | Require sign-off before production release                                                                 | GitHub Environments protection rule (required reviewer)                                   | Formal-verification tooling changes should not go live without human review of test/security results                              |
| 9. Deploy – Production                    | Release the approved build to production with zero/low downtime                                            | Blue-green or rolling deployment via Kubernetes / Render / Railway                        | Minimises downtime risk and enables instant traffic switch-back on failure                                                        |
| 10. Post-Deploy Monitoring & Notification | Confirm health after release and alert the team of issues                                                  | Prometheus/Grafana or platform health checks, Slack/Email webhook notifications           | Early detection of regressions in reachability-analysis latency or API errors                                                     |

### 3. Pipeline Diagram

The diagram below traces the workflow from code commit through to production deployment and monitoring. Feature-branch pushes execute only the fast-feedback stages (Build → Static Analysis → Tests); merges to main run the full chain through to Production.

## CI/CD Pipeline - Formal Verification Framework for E-Transaction Protocols



### 4. Automated Testing Strategy

Testing is layered to match the system's architecture and to keep the fast feedback loop fast while still catching correctness regressions in the automaton/reachability logic before release.

#### 4.1 Unit Tests

- Automaton-construction functions: verify individual protocol automata (card-present, NFC, blockchain) are built with the correct states and transitions for representative inputs.
- Product-automaton composition: verify the synchronised product of two or more automata produces the expected combined state space on small fixture examples.
- Reachability/CTL checker: verify known-good and known-bad transaction sequences are classified as “reachable/safe” or “reachable/violating” correctly.

#### 4.2 Integration Tests

- API layer: verify the REST endpoints correctly invoke the verification engine and return well-formed CTL verification results and fraud-coverage summaries.
- Data layer ↔ engine: verify transaction/protocol definitions loaded from storage are correctly translated into automaton specifications.

#### 4.3 Regression Suite (attack scenarios)

- A fixed library of transaction sequences representing replay attacks, double-spending, identity forgery, and race conditions is re-run on every pipeline execution; a pass requires the framework to flag every scenario in the library.

#### 4.4 End-to-End (E2E) and Load Tests

- Selenium/Playwright scripts exercise the dashboard: uploading a protocol definition, running verification, and viewing the automata diagram and coverage report.
- Locust-based load tests confirm reachability analysis on larger composed automata completes within an acceptable time budget before the build is promoted to staging.

## 5. Deployment Strategy

---

### 5.1 Environments

- Development: local Docker Compose stack used during feature development; not part of the automated pipeline.
- Staging: automatically deployed on every merge to main; mirrors production configuration and is used for smoke tests and manual QA before approval.
- Production: deployed only after the manual approval gate passes; serves the live dashboard and API.

### 5.2 Deployment Method

A blue-green (or rolling, if running on Kubernetes) deployment strategy is used for production releases: the new container image is started alongside the current version, health-checked, and traffic is switched over only once it passes checks — eliminating downtime and allowing an instant switch-back if the new version misbehaves.

## 6. Security, Quality Checks, Notifications, and Rollback

---

### 6.1 Security

- Dependency and container image scanning (Trivy, OWASP Dependency-Check) block the pipeline on high/critical vulnerabilities.
- Static application security testing (CodeQL, Bandit) runs on every pull request.
- Secrets (API keys, registry credentials) are stored in the CI platform's encrypted secret store, never committed to the repository.

### 6.2 Quality Gates

- Minimum code-coverage threshold (e.g., 80% on the verification engine) enforced before merge.
- Required PR review plus all status checks green before merge to main is permitted (branch protection rule).

### 6.3 Notifications

- Slack/email webhook posts build, test, and deployment status to the team channel, including a direct link to failed job logs.
- On production deployment, a summary notification includes the image tag, git SHA, and a rollback command.

### 6.4 Rollback Mechanism

- Every production image is tagged with a semantic version and git SHA and retained in the container registry, so rollback is a single redeploy of the last known-good tag.
- Blue-green deployment keeps the previous version running until the new version is confirmed healthy, enabling an immediate traffic switch-back with zero rebuild time if a post-deploy health check fails.
- Database/schema changes (if any) are written as reversible migrations so a rollback of the application image does not leave the data layer in an incompatible state.

## 7. Summary

---

This CI/CD design automates the path from code commit to production for the Formal Verification Framework, with fast-feedback checks on every branch push and a full build → test → security-scan → package → staging → approval → production → monitor chain on merges to main. The regression suite of known attack scenarios (Section 4.3) is the project-specific safeguard: it ensures that no change to the automaton or reachability logic can silently reduce the framework's fraud-detection coverage without failing the pipeline.